📖 Chapter 3: Express.js

Table of Contents

3.1 What is Express.js?

> ***Definition:*** *Express.js is a minimal and flexible Node.js web application framework that provides robust features for building web and mobile applications. It simplifies routing, middleware handling, and HTTP request/response management, making it the de facto standard for Node.js web servers.*

Key Features:

- Minimalist and unopinionated
- Robust routing system
- Middleware support
- Template engine support
- Easy to extend

3.2 Basic Setup

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

// Built-in middleware to parse JSON body
app.use(express.json());

// Parse URL-encoded data (form submissions)
app.use(express.urlencoded({ extended: true }));

// Serve static files
app.use(express.static('public'));

// Basic route
app.get('/', (req, res) => {
    res.send('Hello World!');
});

// Start server
app.listen(PORT, () => {
```

```
    console.log(`Server running on http://localhost:${PORT}`);
});

// For testing - export app without listening
module.exports = app;
```

3.3 Routing

> ***Definition:*** *Routing refers to determining how an application responds to client requests to specific endpoints (URI path + HTTP method). Express provides methods like* `app.get()` , `app.post()` , `app.put()` , `app.delete()` *that define routes with path patterns and handler functions.*

```
// Sample data
let users = [
    { id: 1, name: "John", email: "john@example.com" },
    { id: 2, name: "Jane", email: "jane@example.com" }
];

// ==================== GET - Retrieve data ====================
// Get all users
app.get('/api/users', (req, res) => {
    res.json(users);
});

// Get single user by ID
app.get('/api/users/:id', (req, res) => {
    const { id } = req.params;
    const user = users.find(u => u.id === parseInt(id));

    if (!user) {
        return res.status(404).json({ message: 'User not found' });
    }
    res.json(user);
});

// Multiple route parameters
app.get('/api/users/:userId/posts/:postId', (req, res) => {
    const { userId, postId } = req.params;
    res.json({ userId, postId });
});

// ==================== POST - Create data ====================
app.post('/api/users', (req, res) => {
    const { name, email } = req.body;

    // Validation
    if (!name || !email) {
        return res.status(400).json({ message: 'Name and email required' });
    }
```

```
    const newUser = {
        id: Date.now(),
        name,
        email
    };
    users.push(newUser);
    res.status(201).json(newUser);
});

// ==================== PUT - Replace entire resource ====================
app.put('/api/users/:id', (req, res) => {
    const { id } = req.params;
    const { name, email } = req.body;
    const index = users.findIndex(u => u.id === parseInt(id));

    if (index === -1) {
        return res.status(404).json({ message: 'User not found' });
    }

    // Replace entire object
    users[index] = { id: parseInt(id), name, email };
    res.json(users[index]);
});

// ==================== PATCH - Partial update ====================
app.patch('/api/users/:id', (req, res) => {
    const { id } = req.params;
    const updates = req.body;
    const index = users.findIndex(u => u.id === parseInt(id));

    if (index === -1) {
        return res.status(404).json({ message: 'User not found' });
    }

    // Merge updates with existing
    users[index] = { ...users[index], ...updates };
    res.json(users[index]);
});

// ==================== DELETE - Remove data ====================
app.delete('/api/users/:id', (req, res) => {
    const { id } = req.params;
    const index = users.findIndex(u => u.id === parseInt(id));

    if (index === -1) {
        return res.status(404).json({ message: 'User not found' });
    }

    users.splice(index, 1);
    res.status(204).send(); // No content
});
```

```
// Route with regex pattern
app.get('/api/files/*', (req, res) => {
    res.send(`Accessing: ${req.params[0]}`);
});

// app.all() - Match all HTTP methods
app.all('/api/secret', (req, res) => {
    res.json({ message: 'This handles any HTTP method' });
});

// Route chaining
app.route('/api/books')
    .get((req, res) => res.json(books))
    .post((req, res) => res.json({ created: req.body }))
    .delete((req, res) => res.json({ deleted: true }));
```

3.4 Middleware

> ***Definition:*** *Middleware functions are functions that have access to the request object (req), response object (res), and the next middleware function (next). They can execute code, modify req/res objects, end the request-response cycle, or call next() to pass control to the next middleware.*

```
// ==================== CUSTOM MIDDLEWARE ====================

// Logger middleware
const logger = (req, res, next) => {
    const timestamp = new Date().toISOString();
    console.log(`[${timestamp}] ${req.method} ${req.url}`);
    next(); // MUST call next() to continue
};

// Apply globally (to all routes)
app.use(logger);

// Apply to specific path
app.use('/api', logger);

// Apply to specific route
app.get('/api/users', logger, (req, res) => {
    res.json(users);
});

// Request timing middleware
const timer = (req, res, next) => {
    req.startTime = Date.now();

    // Run after response is sent
    res.on('finish', () => {
        const duration = Date.now() - req.startTime;
        console.log(`Request took ${duration}ms`);
```

```
    });

    next();
};

// ==================== AUTHENTICATION MIDDLEWARE ====================
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
    // Get token from header
    const authHeader = req.headers.authorization;
    const token = authHeader && authHeader.split(' ')[1]; // "Bearer TOKEN"

    if (!token) {
        return res.status(401).json({ message: 'Access token required' });
    }

    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = decoded; // Attach user to request
        next();
    } catch (error) {
        return res.status(403).json({ message: 'Invalid or expired token' });
    }
};

// Protected routes
app.get('/api/profile', authenticate, (req, res) => {
    res.json({ user: req.user });
});

// ==================== AUTHORIZATION MIDDLEWARE ====================
const authorize = (...roles) => {
    return (req, res, next) => {
        if (!req.user) {
            return res.status(401).json({ message: 'Not authenticated' });
        }

        if (!roles.includes(req.user.role)) {
            return res.status(403).json({ message: 'Not authorized' });
        }

        next();
    };
};

// Only admin can access
app.delete('/api/users/:id', authenticate, authorize('admin'), (req, res) => {
    // Delete user...
});

// ==================== VALIDATION MIDDLEWARE ====================
```

```
const validateUser = (req, res, next) => {
    const { name, email } = req.body;
    const errors = [];

    if (!name || name.length < 2) {
        errors.push('Name must be at least 2 characters');
    }

    if (!email || !email.includes('@')) {
        errors.push('Valid email required');
    }

    if (errors.length > 0) {
        return res.status(400).json({ errors });
    }

    next();
};

app.post('/api/users', validateUser, (req, res) => {
    // Create user...
});

// ==================== THIRD-PARTY MIDDLEWARE ====================
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');

// CORS - Cross-Origin Resource Sharing
app.use(cors()); // Allow all origins
app.use(cors({
    origin: 'http://localhost:3000',
    methods: ['GET', 'POST'],
    credentials: true
}));

// Helmet - Security headers
app.use(helmet());

// Morgan - HTTP request logger
app.use(morgan('dev'));     // Concise dev format
app.use(morgan('combined')); // Apache combined format

// Rate limiting
const limiter = rateLimit({
    windowMs: 15 * 60 * 1000, // 15 minutes
    max: 100, // Limit each IP to 100 requests per window
    message: 'Too many requests, please try again later'
});
app.use('/api', limiter);
```

```
// ==================== ASYNC MIDDLEWARE WRAPPER ====================
const asyncHandler = (fn) => (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
};

// Usage - no need for try/catch
app.get('/api/users', asyncHandler(async (req, res) => {
    const users = await User.find();
    res.json(users);
}));
```

3.5 Request Object

> ***Definition:*** *The* `req` *(request) object represents the HTTP request from the client. It contains properties for query strings, parameters, body, headers, cookies, and more. Express enhances the native Node.js request object with additional helpful properties.*

```
app.post('/api/users/:id', (req, res) => {
    // URL Parameters - /api/users/123
    console.log(req.params.id);  // "123"

    // Query String - /api/users?page=1&limit=10
    console.log(req.query.page);   // "1"
    console.log(req.query.limit);  // "10"

    // Request Body (requires express.json() middleware)
    console.log(req.body);  // { name: "John", email: "..." }

    // Headers
    console.log(req.headers);                    // All headers
    console.log(req.headers['content-type']);    // 'application/json'
    console.log(req.headers.authorization);      // 'Bearer token...'
    console.log(req.get('Content-Type'));        // Same as above

    // HTTP Method
    console.log(req.method);  // "POST"

    // URL Info
    console.log(req.url);           // "/api/users/123?page=1"
    console.log(req.path);          // "/api/users/123"
    console.log(req.originalUrl);   // Original URL before rewrites
    console.log(req.baseUrl);       // Router base path
    console.log(req.hostname);      // "localhost"
    console.log(req.protocol);      // "http" or "https"
    console.log(req.secure);        // true if HTTPS

    // IP Address
    console.log(req.ip);            // Client IP
    console.log(req.ips);           // Array if behind proxy
```

```
    // Cookies (requires cookie-parser middleware)
    console.log(req.cookies);       // { sessionId: "abc123" }

    // Check content type
    console.log(req.is('json'));    // 'json' or false
    console.log(req.is('html'));    // 'html' or false

    // Custom properties (set by middleware)
    console.log(req.user);          // Set by auth middleware
});

// Pagination example
app.get('/api/users', (req, res) => {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;
    const sort = req.query.sort || 'name';
    const order = req.query.order === 'desc' ? -1 : 1;
    const search = req.query.search || '';

    // Use these for database query...
});
```

3.6 Response Object

> ***Definition:*** *The* `res` *(response) object represents the HTTP response that Express sends back to the client. It provides methods for setting status codes, headers, cookies, and sending various types of content (JSON, HTML, files, etc.).*

```
app.get('/api/demo', (req, res) => {
    // ==================== SENDING RESPONSES ====================

    // Send JSON (most common for APIs)
    res.json({ name: 'John', age: 30 });

    // Send with status code
    res.status(201).json({ message: 'Created' });

    // Send plain text
    res.send('Hello World');

    // Send HTML
    res.send('<h1>Hello World</h1>');

    // Send status only
    res.sendStatus(204);  // 204 No Content
    // Equivalent to: res.status(204).send()

    // ==================== STATUS CODES ====================
    res.status(200);  // OK
    res.status(201);  // Created
```

```
    res.status(204);  // No Content
    res.status(400);  // Bad Request
    res.status(401);  // Unauthorized
    res.status(403);  // Forbidden
    res.status(404);  // Not Found
    res.status(500);  // Internal Server Error

    // ==================== HEADERS ====================
    res.set('Content-Type', 'application/json');
    res.set('X-Custom-Header', 'value');
    res.set({
        'Content-Type': 'text/plain',
        'X-Another-Header': 'value'
    });

    // ==================== COOKIES ====================
    res.cookie('token', 'abc123', {
        httpOnly: true,     // Not accessible via JavaScript
        secure: true,       // HTTPS only
        maxAge: 3600000,    // 1 hour in ms
        sameSite: 'strict'  // CSRF protection
    });

    res.clearCookie('token');

    // ==================== REDIRECTS ====================
    res.redirect('/new-url');
    res.redirect(301, '/permanent-new-url');
    res.redirect('back');  // Redirect to referrer

    // ==================== FILES ====================
    // Send file
    res.sendFile('/absolute/path/to/file.pdf');
    res.sendFile('file.pdf', { root: './public' });

    // Download file (prompts save dialog)
    res.download('/path/to/file.pdf');
    res.download('/path/to/file.pdf', 'custom-name.pdf');

    // ==================== TEMPLATES ====================
    // Render view template (requires view engine setup)
    res.render('index', { title: 'Home', user: 'John' });

    // ==================== CHAINING ====================
    res
        .status(201)
        .set('X-Custom', 'value')
        .cookie('session', 'abc')
        .json({ success: true });
});
```

3.7 Router (Modular Routes)

> ***Definition:*** *Express Router is a mini-application that handles only routes and middleware. It allows you to organize routes into separate files/modules, making code modular, maintainable, and easier to test in larger applications.*

```
// ==================== routes/userRoutes.js ====================
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const { authenticate, authorize } = require('../middleware/auth');

// All routes here are prefixed with /api/users (set in app.js)

// GET /api/users
router.get('/', userController.getAllUsers);

// GET /api/users/:id
router.get('/:id', userController.getUserById);

// POST /api/users
router.post('/', authenticate, userController.createUser);

// PUT /api/users/:id
router.put('/:id', authenticate, userController.updateUser);

// DELETE /api/users/:id (admin only)
router.delete('/:id', authenticate, authorize('admin'), userController.deleteUser);

// Router-level middleware (applies to all routes in this router)
router.use((req, res, next) => {
    console.log('User routes accessed');
    next();
});

module.exports = router;

// ==================== routes/authRoutes.js ====================
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');

router.post('/register', authController.register);
router.post('/login', authController.login);
router.post('/logout', authController.logout);
router.post('/forgot-password', authController.forgotPassword);
router.post('/reset-password/:token', authController.resetPassword);

module.exports = router;

// ==================== app.js ====================
const express = require('express');
```

```
const app = express();

// Import routes
const userRoutes = require('./routes/userRoutes');
const authRoutes = require('./routes/authRoutes');
const productRoutes = require('./routes/productRoutes');

// Middleware
app.use(express.json());

// Mount routes
app.use('/api/users', userRoutes);     // /api/users/*
app.use('/api/auth', authRoutes);      // /api/auth/*
app.use('/api/products', productRoutes); // /api/products/*

// 404 handler (after all routes)
app.use((req, res) => {
    res.status(404).json({ message: 'Route not found' });
});

module.exports = app;

// ==================== controllers/userController.js ====================
const User = require('../models/User');

exports.getAllUsers = async (req, res, next) => {
    try {
        const users = await User.find();
        res.json({ count: users.length, data: users });
    } catch (error) {
        next(error);
    }
};

exports.getUserById = async (req, res, next) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }
        res.json(user);
    } catch (error) {
        next(error);
    }
};

// ... other controller methods
```

3.8 Error Handling

> ***Definition:*** *Express error handling uses middleware with four parameters (err, req, res, next). Errors can be passed to the error handler using* `next(error)`*. Proper error handling ensures consistent error responses and prevents server crashes.*

```
// ==================== Custom Error Class ====================
class AppError extends Error {
    constructor(message, statusCode) {
        super(message);
        this.statusCode = statusCode;
        this.status = `${statusCode}`.startsWith('4') ? 'fail' : 'error';
        this.isOperational = true; // Known operational error

        Error.captureStackTrace(this, this.constructor);
    }
}

// ==================== Async Handler ====================
const asyncHandler = (fn) => (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
};

// ==================== Route Using Error Handling ====================
app.get('/api/users/:id', asyncHandler(async (req, res, next) => {
    const user = await User.findById(req.params.id);

    if (!user) {
        return next(new AppError('User not found', 404));
    }

    res.json(user);
}));

// ==================== 404 Handler ====================
// Must be after all routes
app.use((req, res, next) => {
    next(new AppError(`Cannot find ${req.originalUrl}`, 404));
});

// ==================== Global Error Handler ====================
// Must have 4 parameters, must be last middleware
app.use((err, req, res, next) => {
    err.statusCode = err.statusCode || 500;
    err.status = err.status || 'error';

    // Development: Send full error
    if (process.env.NODE_ENV === 'development') {
        res.status(err.statusCode).json({
            status: err.status,
            error: err,
            message: err.message,
            stack: err.stack
```

```
        });
    }
    // Production: Send minimal error
    else {
        // Operational error: send message
        if (err.isOperational) {
            res.status(err.statusCode).json({
                status: err.status,
                message: err.message
            });
        }
        // Programming error: don't leak details
        else {
            console.error('ERROR 💥', err);
            res.status(500).json({
                status: 'error',
                message: 'Something went wrong'
            });
        }
    }
});

// ==================== Handling Specific Errors ====================
const handleDBError = (err) => {
    // MongoDB duplicate key
    if (err.code === 11000) {
        const field = Object.keys(err.keyValue)[0];
        return new AppError(`${field} already exists`, 400);
    }

    // MongoDB validation error
    if (err.name === 'ValidationError') {
        const messages = Object.values(err.errors).map(e => e.message);
        return new AppError(messages.join('. '), 400);
    }

    // MongoDB cast error (invalid ID)
    if (err.name === 'CastError') {
        return new AppError(`Invalid ${err.path}: ${err.value}`, 400);
    }

    // JWT errors
    if (err.name === 'JsonWebTokenError') {
        return new AppError('Invalid token', 401);
    }
    if (err.name === 'TokenExpiredError') {
        return new AppError('Token expired', 401);
    }

    return err;
};
```

```
// ==================== Unhandled Rejections & Exceptions ====================
// Unhandled promise rejections
process.on('unhandledRejection', (err) => {
    console.error('UNHANDLED REJECTION! 💥 Shutting down...');
    console.error(err.name, err.message);
    server.close(() => process.exit(1));
});

// Uncaught exceptions
process.on('uncaughtException', (err) => {
    console.error('UNCAUGHT EXCEPTION! 💥 Shutting down...');
    console.error(err.name, err.message);
    process.exit(1);
});
```

Next: [Chapter 4 - MongoDB & Mongoose]